EXPRESS + FASTIFY + IoC

Від handle() до контейнера: middleware, хуки, схеми, InversifyJS

Заняття №4

Node.js Pro

Лектор
Кирило Орлов

r_d

СТЕК НА СЬОГОДНІ: EXPRESS · ROUTER · FASTIFY · SCHEMA · INVERSIFY

Перша третина — Express руками	Друга третина — Fastify і IoC
handle() з L#3 — звідки стартуємо	fastify — іменований життєвий цикл
міні-Express — масив функцій + next()	schema — валідація + серіалізація
express — Router, error middleware	inversify — контейнер збирає граф
Express 5 — async-помилки йдуть самі	Express = масив функцій. Магії немає

НАСКРІЗНА НИТКА
Та сама драбинка, що на L#3: сирі байти → net → http → Express → Fastify. Кожен щабель прибирає ручну роботу й додає магії — в кінці ми її розберемо.

ПЛАН ЗАНЯТТЯ



- Виправлення з минулої лекції (4 хв)
- Міні-Express у 50 рядках: масив + next()
- Router, error middleware, async-помилки
- Fastify: життєвий цикл і хуки
- Схеми: валідація + серіалізація; плагіни
- IoC та InversifyJS: розбираємо магію декораторів

ВИПРАВЛЕННЯ З МИНУЛОЇ ЛЕКЦІЇ

01. certbot

видає сертифікат ВІД Let's Encrypt — це не self-signed

02. x509

це ФОРМАТ сертифіката, а не «самопідписаний»

03. SAN / CAA

SAN — розширення СЕРТИФІКАТА. CAA — оце вже DNS-запис

04. TLS 1.2

клієнт шифрує ПУБЛІЧНИМ ключем, і число там одне

05. ECDHE

Elliptic Curve Diffie–Hellman Ephemeral

06. 6-денні серти

окремий ACME-профіль, а не «домовитись з Let's Encrypt»

ПРОБЛЕМА: handle() НЕ МАСШТАБУЄТЬСЯ

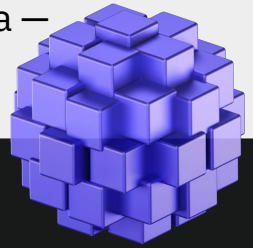
На L#3 увесь роутинг був один if / switch у handle()

додаємо auth → checkAuth() у кожній гілці

додаємо логування → log() у кожній гілці

cross-cutting concerns розповзаються по всіх обробниках

Потрібен КОНВЕЄР, а не дерево умов. Кожна турбота — один раз, у своїй ланці. Саме це й робить Express.



```
ДЕРЕВО УМОВ

handle()
 ├─ /users
 │    auth  •
 │    log   •
 ├─ /admin
 │    auth  •
 │    log   •
 └─ /health
      auth  •
      log   •

 • одне й те саме,
   написане 6 разів


КОНВЕЄР

 log → auth → route
           → handler

 кожна турбота
 рівно один раз
```

MINI-EXPRESS: 50 РЯДКІВ, І МАГІЇ НЕМАЄ

```
import http from 'node:http';
const stack = [];              // увесь Express — оце
const use = (fn) => stack.push(fn);   // реєстрація = push

function run(req, res) {       // виконання ланцюга
  let i = -1;
  const next = () => {
    i += 1;                    // рухаємо індекс
    const fn = stack[i];
    if (fn) fn(req, res, next); // передаємо next далі
  };
  next();
}
use((req, res, next) => { console.log('→ log'); next(); });
use((req, res, next) => {      // ланка може обірвати ланцюг
  if (!req.headers.authorization)
    return res.writeHead(401).end('no token\n');
  console.log('→ auth'); next();
});
use((req, res) => { console.log('→ handler'); res.end('ok\n'); });

http.createServer(run).listen(3000);  // run — застосунок
```

Увесь секрет — масив функцій і один індекс

1 · use(fn) — це просто push у масив, більше нічого

2 · next() — викликати наступну функцію за індексом

3 · не викликав next() — ланцюг обірвався, і це нормально

curl /users → log → auth → handler → 200

урок: next — не ключове слово, а наш власний аргумент



«КОНВЕЄР НА ЗАВОДІ»: EXPRESS ЯК АНАЛОГІЯ

ЧАСТИНА EXPRESS	АНАЛОГІЯ (КОНВЕЄР)
req — об'єкт запиту	деталь, що їде конвеєром
middleware — ланка ланцюга	станція на стрічці
next() — передати далі	штовхнув деталь далі
res.send() — відповісти й зупинити	зняв деталь зі стрічки
error middleware (4 аргументи)	окремий стіл для браку в кінці

Де аналогія ламається: справжній конвеєр не вміє стрибати одразу до столу браку, а next(err) — уміє. Порядок станцій задано при монтажі, не під час роботи.



ПОРЯДОК РЕЄСТРАЦІЇ = ПОРЯДОК ВИКОНАННЯ

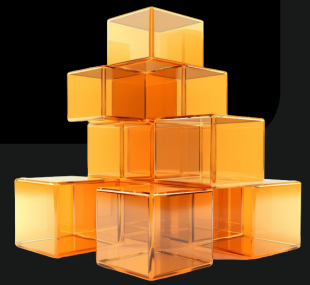
- app.use і app.get — ОДИН спільний список, не дві групи
- виконується строго згори вниз по файлу
- next() — далі по ланцюгу · next(err) — стрибок в обробники помилок
- res.send() без next() — ланцюг завершено, решта не виконається
- типова помилка: думати, що всі use ідуть перед усіма get
- express.Router() — той самий список, змонтований на префікс

```
ОДИН СПИСОК, НЕ ДВІ ГРУПИ

app.use(a)  ┐
app.get(b)  │
app.use(c)  ├→  stack = [a,b,c,d,e]
app.get(d)  │
app.use(e)  ┘

i = -1
next() → i += 1 → stack[i](req,res,next)

не викликав next()
    →  і не рухається
    →  ланцюг стоїть
```





```
// Express: те саме, що ми щойно написали руками

import express from 'express';
const app = express();
app.use(express.json());          // парсер тіла — окремою ланкою
app.use((req, res, next) => {
  console.log(req.method, req.path);  // логер бачить УСІ запити
  next();                             // не викликав — ланцюг стане
});

app.get('/users/:id', (req, res) => res.json({ id: req.params.id }));
app.get('/health', (req, res) => res.send('ok'));
app.listen(3000);
```

```
// Router: міні-застосунок зі своїм стеком, змонтований на префікс

import express, { Router } from 'express';
const users = Router();

users.use((req, res, next) => {              // auth на ВСЮ гілку
  if (!req.headers.authorization) return res.sendStatus(401);
  next();
});

users.get('/:id', (req, res) => res.json({   // шлях ВІДНОСНИЙ
  url: req.url,                  // '/42'
  originalUrl: req.originalUrl, // '/api/users/42'
}));

express().get('/health', (r, s) => s.send('ok'))  // поза гілкою — без auth
         .use('/api/users', users).listen(3000);  // ← префікс ТУТ
```



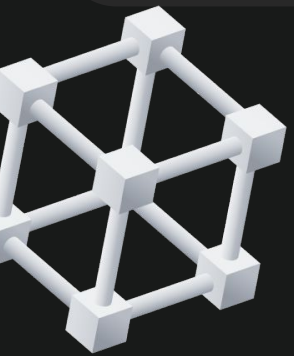
ERROR MIDDLEWARE: АРНІСТЬ — ЦЕ КОНТРАКТ

обробник помилок = функція з РІВНО чотирма параметрами

усередині Express: if (fn.length !== 4) → це не обробник помилок

прибрав невикористаний next → МОВЧКИ перестав ловити помилки

реєструється ОСТАННІМ. Обробників може бути кілька: вузький → next(err) → загальний. У проді клієнту — без стеку, у лог — повний





```
// Express 5: async-помилка йде в next(err) САМА

import express from 'express';
const app = express();
app.get('/user/:id', async (req, res) => {   // без try/catch
  const user = await db.findUser(req.params.id);  // кине, якщо нема
  res.json(user);
});

app.use((err, req, res, next) => {           // рівно 4 параметри
  res.status(404).json({ error: err.message });
});
// Express 4: той самий код → запит висить до таймауту
```

ASYNC-ПОМИЛКИ: EXPRESS 4 ПРОТИ 5

Express 4

відхилений проміс губиться → запит висить до таймауту

Express 5.2

роутер сам робить ret.then(null, err => next(err))

Не ловиться ніде

throw у setTimeout — поза стеком роутера, не ловить ніхто

Чому це важливо саме зараз
Більшість гайдів, StackOverflow і шаблонів у мережі досі описують Express 4 — там був потрібен пакет express-async-errors або власна обгортка asyncHandler. Ми на express@5.2.1: обгортки більше не потрібні, і копіювати їх у новий проєкт — тягнути зайве.

EXPRESS 5 ПРОТИ FASTIFY 5: ЧЕСНА ТАБЛИЦЯ

Аспект	Express 5.2	Fastify 5.11
Розширення	плаский конвеєр app.use	іменований цикл + плагіни
Ізоляція	немає — app.use глобальний	плагін інкапсульований
Валідація	окремий шар, руками	вбудована: schema.body → ajv
Серіалізація	окремий шар, руками	скомпільована: schema.response
Async-помилки	ловить сам — але тільки з 5-ї версії	ловить сам — від першої версії

FASTIFY: ІНША МОДЕЛЬ, А НЕ «ТЕ САМЕ ШВИДШЕ»

01. Express — плаский конвеєр: стан залежить від порядку рядків

02. Fastify — іменований цикл: стан гарантований структурою

03. request / reply замість req / res — вони НЕ сумісні

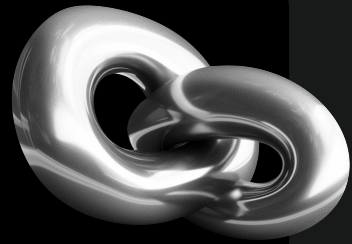
04. схема — не «валідація на додачу», а джерело скомпільованого коду

05. Express-middleware тут не працює без адаптера @fastify/middie

ЖИТТЄВИЙ ЦИКЛ FASTIFY: ДЕ ЩО ВЖЕ ДОСТУПНО

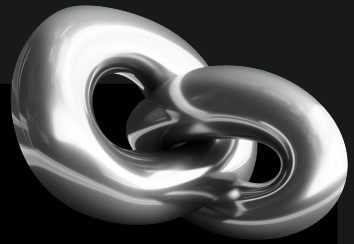
```
onRequest        auth, rate-limit          body: undefined
     ↓
preParsing       сирий потік               body: undefined
     ↓
  ── ТУТ ПАРСИТЬСЯ ТІЛО ──────────────────────────────
     ↓
preValidation    нормалізація              body: є
     ↓
preHandler       бізнес-перевірки          body: є
     ↓
  handler        ваш код
     ↓
preSerialization зміна об'єкта відповіді
     ↓
onSend           заголовки, сира відповідь
     ↓
onResponse       метрики, elapsedTime      відповідь пішла

onError — окремо, спрацьовує з будь-якого етапу
```



ХУКИ FASTIFY: ДЕ ТІЛО ВЖЕ Є, А ДЕ ЩЕ НІ

Хук	Для чого	request.body
onRequest	auth, rate-limit — відкидаємо ДО парсингу	undefined
preParsing	втручання у сирий потік	undefined
preValidation	нормалізація тіла до перевірки схемою	є
preHandler	бізнес-перевірки, завантаження користувача	є
onResponse	метрики, час відповіді	відповідь уже пішла



```
// Fastify: схема валідує вхід і ріже вихід

import Fastify from 'fastify';
const app = Fastify({ logger: true });

app.get('/user/:id', {
  schema: { response: { 200: { type: 'object', properties: {
    id: { type: 'string' }, name: { type: 'string' },
  } } } },
}, async (request) => ({
  id: request.params.id, name: 'Kyrylo',
  passwordHash: 'secret',   // ← у відповідь НЕ потрапить: немає у схемі
}));

await app.listen({ port: 3000 });
```

СХЕМИ Й ПЛАГІНИ: ЗВІДКИ ПЕРЕВАГА FASTIFY

schema.body → ajv компілює перевірку входу в код

schema.response → fast-json-stringify компілює серіалізацію

перевага НЕ з «швидшого роутера», а зі зникнення JSON.stringify

fastify.register(plugin) — усе всередині в ОКРЕМОМУ контексті

плагін бачить те, що вище; назовні не витікає. app.use — навпаки

потрібно глобально — обгорни у fastify-plugin (fp)

«КУХНЯ РЕСТОРАНУ»: IoC ЯК АНАЛОГІЯ

ЧАСТИНА IoC

- сервіс, якому потрібні залежності
- token — Symbol.for('Logger')
- container.bind(TOKEN).to(Class)
- container.get(TOKEN) → готовий об'єкт з усіма залежностями

АНАЛОГІЯ (КУХНЯ)

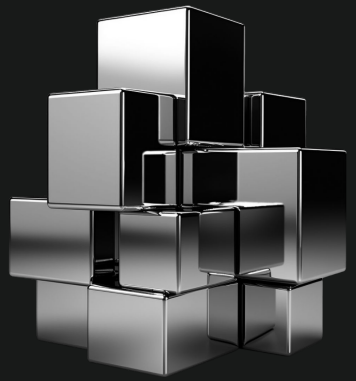
- кухар, якому потрібні продукти
- назва позиції в меню
- рецепт: «позиція X готується так»
- кухар не бігає по магазинах — каже назву, і йому приносять усе готове

IoC: ЧОМУ ГРАФ ЗАЛЕЖНОСТЕЙ ЗБИРАЮТЬ НЕ РУКАМИ

Ручна збірка: new UserService(repo, logger, mailer, config) — додав параметр у конструктор, правиш УСІ місця створення

IoC — це інверсія: не ти створюєш залежність, а тобі її дають. Контейнер знає рецепти й збирає граф рекурсивно

⚠ Ціна — непрозорість: помилка резолву вилазить у рантаймі, а не при компіляції. Плюс керує часом життя: Singleton / Transient / Request



```
// InversifyJS 8: контейнер збирає граф залежностей
import 'reflect-metadata';   // ← ПЕРШИМ рядком, інакше падіння в рантаймі
import { Container, injectable, inject } from 'inversify';
const TYPES = { Logger: Symbol.for('Logger') };

@injectable('Singleton')     // скоуп в декораторі — нове у v8
class ConsoleLogger { log(m) { console.log(m); } }

@injectable()
class UserService { constructor(@inject(TYPES.Logger) private log) {} }

const c = new Container();
c.bind(TYPES.Logger).to(ConsoleLogger);
c.bind(UserService).toSelf();
c.get(UserService);          // граф зібрано рекурсивно
```

ПІДСУМКИ + МІСТ ДО L#5 І L#6

EXPRESS

Express — це масив функцій і next(). Магії немає

FASTIFY

інша модель: іменований цикл + скомпільовані зі схеми валідація й серіалізація

далі

L#5 Docker — усе це поїде в контейнер · L#6 NestJS — ті самі декоратори

Тест #4

2 бали · порядок middleware · арність error-обробника · async 4 vs 5 · хуки Fastify · @injectable / @inject · коли Fastify > Express

IoC

контейнер збирає граф залежностей і керує часом життя